

Algoritmos Lineares: Counting Sort e Radix Sort

Os algoritmos **Counting Sort** e **Radix Sort** são chamados de **algoritmos lineares** porque, em determinadas condições, conseguem ordenar elementos em tempo próximo de **$O(n)$** , sendo mais rápidos que algoritmos de comparação como Quick Sort, Merge Sort e Heap Sort (que normalmente possuem complexidade $O(n \log n)$).

Counting Sort

1. Counting Sort

Conceito

O **Counting Sort** não compara elementos entre si.

Ele funciona contando quantas vezes cada valor aparece no vetor e depois reconstruindo o vetor ordenado.

É ideal quando:

- Os valores são inteiros.
- O maior valor não é muito grande.
- Existe uma faixa conhecida de valores.

Counting Sort

Exemplo

Vetor:

[4, 2, 2, 8, 3, 3, 1]

Passo 1: Encontrar o maior elemento

Maior valor:

8

Criamos um vetor de contagem com posições de 0 a 8.

Índice: 0 1 2 3 4 5 6 7 8

Contagem: 0 0 0 0 0 0 0 0 0

Counting Sort

Passo 2: Contar ocorrências

Percorremos o vetor original:

4 → conta[4]++

2 → conta[2]++

2 → conta[2]++

8 → conta[8]++

3 → conta[3]++

3 → conta[3]++

1 → conta[1]++

Resultado:

Índice: 0 1 2 3 4 5 6 7 8

Contagem: 0 1 2 2 1 0 0 0 1

Counting Sort

Passo 3: Reconstruir o vetor

Lemos o vetor de contagem:

1 aparece 1 vez

2 aparece 2 vezes

3 aparece 2 vezes

4 aparece 1 vez

8 aparece 1 vez

Resultado:

[1, 2, 2, 3, 3, 4, 8]

COUNTING SORT – PASSO A PASSO

VETOR ORIGINAL:

4

2

2

8

3

3

1

1

ENCONTRAR O MAIOR ELEMENTO

Maior elemento do vetor = 8

Vamos criar um vetor de contagem com índices de 0 até 8.

ÍNDICE

0

1

2

3

4

5

6

8

CONTAGEM

0

0

0

0

0

0

0

0

2

CONTAR AS OCORRÊNCIAS DE CADA ELEMENTO

Percorremos o vetor original e incrementamos a contagem da posição correspondente a cada valor.

ÍNDICE

0

1

2

3

4

5

6

8

CONTAGEM

0

1

2

2

1

0

0

1



1 aparece
1 vez



2 aparece
2 vezes



3 aparece
2 vezes



4 aparece
1 vez



8 aparece
1 vez

3

RECONSTRUIR O VETOR ORDENADO

Percorremos o vetor de contagem da esquerda para a direita. Para cada índice, colocamos no vetor original o índice tantas vezes quanto sua contagem indica.

ÍNDICE (i)

0

1

2

3

4

5

6

7

8

CONTAGEM[i]

0

1

2

2

1

0

0

0

1

AÇÃO

nada

colocar 1
1 vez

colocar 2
2 vezes

colocar 3
2 vezes

colocar 4
1 vez

nada

nada

nada

colocar 8
1 vez

VETOR EM
CONSTRUÇÃO

- - - - -



1 - - - - -



1 2 2 - - - -



1 2 2 3 3 - -



1 2 2 3 3 4 -



1 2 2 3 3 4 8

4

VETOR ORDENADO (RESULTADO FINAL)

1

2

2

3

3

4

8



OBSERVAÇÕES:

- O Counting Sort é um algoritmo estável.
- Ele não compara elementos, apenas conta quantas vezes cada valor aparece.
- Complexidade:** $O(n + k)$, onde n é a quantidade de elementos e k é o maior valor do vetor (neste caso, 8).

Counting Sort

```
#include <stdio.h>
```

```
int getMax(int vetor[], int n) {  
    int max = vetor[0];  
    for(int i = 1; i < n; i++)  
        if(vetor[i] > max)  
            max = vetor[i];  
    return max;  
}
```


Counting Sort

```
void countingSort(int vetor[], int n, int exp) {  
    int saida[n];  
    int contagem[10] = {0};  
    // Contagem dos dígitos  
    for(int i = 0; i < n; i++)  
        contagem[(vetor[i] / exp) % 10]++;  
    // Soma acumulada  
    for(int i = 1; i < 10; i++)  
        contagem[i] += contagem[i - 1];  
    // Construção do vetor ordenado  
    for(int i = n - 1; i >= 0; i--) {  
        int indice = (vetor[i] / exp) % 10;  
        saida[contagem[indice] - 1] = vetor[i];  
        contagem[indice]--;  
    }  
    // Copiar para o vetor original  
    for(int i = 0; i < n; i++)  
        vetor[i] = saida[i];  
}
```


Counting Sort

```
int main() {  
    int vetor[] = {4, 2, 2, 8, 3, 3, 1};  
  
    int n = sizeof(vetor)/sizeof(vetor[0]);  
  
    countingSort(vetor, n);  
  
    printf("Vetor ordenado:\n");  
  
    for(int i = 0; i < n; i++)  
        printf("%d ", vetor[i]);  
  
    return 0;  
}
```

Radix Sort

2. Radix Sort

Conceito

O **Radix Sort** ordena os números dígito por dígito.

Ele normalmente utiliza o **Counting Sort** como algoritmo auxiliar.

A ordenação começa pelo:

Dígito das unidades

Dígito das dezenas

Dígito das centenas

E assim por diante

Exemplo

Vetor:

[170, 45, 75, 90, 802, 24, 2, 66]

Radix Sort

Passo 1: Ordenar pelas unidades

170 → 0

45 → 5

75 → 5

90 → 0

802 → 2

24 → 4

2 → 2

66 → 6

Resultado:

[170, 90, 802, 2, 24, 45, 75, 66]

Radix Sort

Passo 2: Ordenar pelas dezenas

170 → 7

90 → 9

802 → 0

2 → 0

24 → 2

45 → 4

75 → 7

66 → 6

Resultado:

[802, 2, 24, 45, 66, 170, 75, 90]

Radix Sort

Passo 3: Ordenar pelas centenas

802 → 8

2 → 0

24 → 0

45 → 0

66 → 0

170 → 1

75 → 0

90 → 0

Resultado final:

[2, 24, 45, 66, 75, 90, 170, 802]

Radix Sort

Como o Radix Sort funciona visualmente

Vetor Original

170 45 75 90 802 24 2 66

↓ Unidades

170 90 802 2 24 45 75 66

↓ Dezenas

802 2 24 45 66 170 75 90

↓ Centenas

2 24 45 66 75 90 170 802

VETOR ORIGINAL

170	45	75	90	802	24	2	66
-----	----	----	----	-----	----	---	----

Vamos ordenar da direita para a esquerda (unidades, dezenas, centenas) usando Counting Sort em cada etapa.

1ª PASSAGEM – ORDENAR PELO DÍGITO DAS UNIDADES

Vetores e dígitos (unidades)	170	45	75	90	802	24	2	66
	0	5	5	0	2	4	2	6

Dígito das unidades	0	5	5	0	2	4	2	6
---------------------	---	---	---	---	---	---	---	---

↓ Após ordenar pelo dígito das unidades:

	170	90	802	2	24	45	75	66
Unidades	0	0	2	2	4	5	5	6

2ª PASSAGEM – ORDENAR PELO DÍGITO DAS DEZENAS

Vetores e dígitos (dezenas)	170	90	802	2	24	45	75	66
	7	9	0	0	2	4	7	6

Dígito das dezenas	7	9	0	0	2	4	7	6
--------------------	---	---	---	---	---	---	---	---

↓ Após ordenar pelo dígito das dezenas:

	802	2	24	45	66	170	75	90
Dezenas	0	0	2	4	6	7	7	9

3ª PASSAGEM – ORDENAR PELO DÍGITO DAS CENTENAS

Vetores e dígitos (centenas)	802	2	24	45	66	170	75	90
	8	0	0	0	0	1	0	0

Dígito das centenas	8	0	0	0	0	1	0	0
---------------------	---	---	---	---	---	---	---	---

↓ Após ordenar pelo dígito das centenas:

	2	24	45	66	75	90	170	802
Centenas	0	0	0	0	0	0	1	8

VETOR FINAL ORDENADO

2	24	45	66	75	90	170	802
---	----	----	----	----	----	-----	-----



Resumo: O Radix Sort ordena os números dígito por dígito, da direita para a esquerda. A cada passagem, os números ficam ordenados com base naquele dígito, mas a ordenação final só está correta após a última passagem (centenas, no exemplo).

Radix Sort

```
#include <stdio.h>
```

```
int getMax(int vetor[], int n) {  
    int max = vetor[0];  
    for(int i = 1; i < n; i++)  
        if(vetor[i] > max)  
            max = vetor[i];  
    return max;  
}
```

Radix Sort

```
void radixSort(int vetor[], int n) {  
    int max = getMax(vetor, n);  
  
    for(int exp = 1; max / exp > 0; exp *= 10)  
        countingSort(vetor, n, exp);  
}  
  
int main() {  
    int vetor[] = {170, 45, 75, 90, 802, 24, 2, 66};  
    int n = sizeof(vetor)/sizeof(vetor[0]);  
    radixSort(vetor, n);  
    printf("Vetor ordenado:\n");  
    for(int i = 0; i < n; i++)  
        printf("%d ", vetor[i]);  
    return 0;  
}
```

Algoritmos Lineares: Counting Sort e Radix Sort

Resumo Comparativo

Característica	Counting Sort	Radix Sort
Compara elementos?	Não	Não
Baseado em contagem?	Sim	Sim
Usa Counting Sort?	Não	Sim
Funciona melhor com	Faixa pequena de valores	Números inteiros com vários dígitos
Complexidade	$O(n + k)$	$O(d \times (n + k))$
Estável	Sim	Sim (quando usa Counting Sort estável)